

AI Agent for Dimension Reduction and Exploratory Data Analysis

Cassi Chen

1 Introduction and Agent Architecture

Dimension-reduction analyses require dataset-dependent decisions about preprocessing, method selection, hyperparameters, evaluation, and interpretation. Rather than constructing a fixed pipeline that applies the same procedure to every dataset, this project develops an AI agent that performs dimension reduction and exploratory data analysis (EDA) with minimal human guidance. Given a dataset and its documentation, the agent inspects the data, determines appropriate preprocessing, selects dimension-reduction methods, chooses method settings, executes statistical tools, evaluates the resulting representations, performs targeted follow-up when warranted, and generates a final analysis report.

The system separates **reasoning and orchestration** from **statistical computation**. Codex serves as the reasoning agent. It plans the analysis, makes preprocessing and method-selection decisions, chooses tool arguments, interprets outputs, and determines subsequent actions. Local Python programs provide deterministic tools for data inspection, preprocessing, dimension reduction, embedding evaluation, and visualization. This separation allows the agent to make adaptive decisions while keeping the underlying numerical computations reproducible.

Two layers of instructions guide Codex. **AGENTS.md** defines the repository-wide workflow and constraints, including train/evaluation separation, label handling, method selection, evaluation, and reporting. Specialized **SKILL.md** files provide statistical guidance for data inspection, preprocessing, individual dimension-reduction methods, and embedding evaluation. Skills describe when methods are appropriate and how their outputs should be interpreted without prescribing the same analysis for every dataset.

The available dimension-reduction tools are PCA, Kernel PCA, MDS, Isomap, locally linear embedding (LLE), Laplacian Eigenmaps, t-SNE, and UMAP. The agent considers the available methods but selects only a subset appropriate for the observed data and analysis objective. The Python tools also support three preprocessing choices: preservation of the supplied representation, feature-wise standardization, and library-size normalization followed by $\log_{1p}$ transformation.

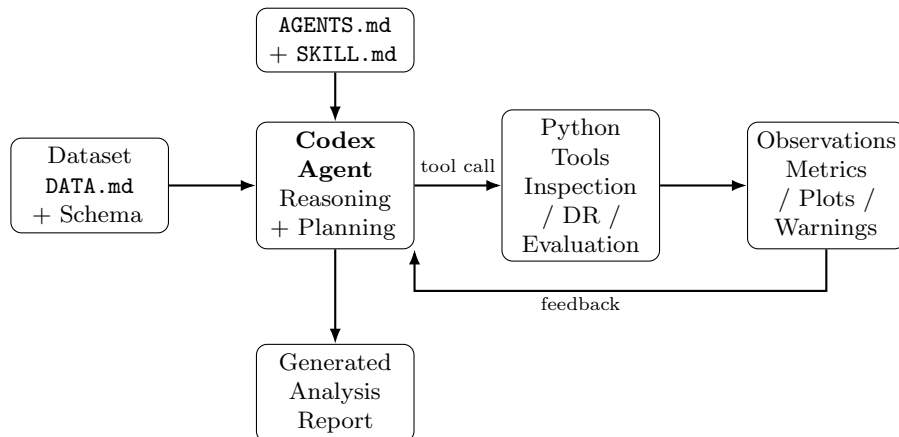


Figure 1: Agent architecture. Codex selects and invokes deterministic analysis tools, observes their outputs, and uses those observations to determine subsequent actions and generate the final report.

The overall agent design was inspired by the course lecture architecture for tool-using coding agents, in which an LLM selects actions, local tools perform deterministic computation, and resulting observations are returned to the agent to guide subsequent decisions [1].

2 Agent Decision-Making and Implementation

The agent follows an iterative *inspect-decide-execute-evaluate-adapt* workflow. It first reads the dataset documentation and schema and invokes the data-inspection tool. The resulting profile describes dimensions, feature types, missing and

nonfinite values, sparsity, feature scales, and other structural characteristics. These observations are then combined with the preprocessing skill to select an explicit preprocessing strategy. Parameters learned across observations, such as feature means and standard deviations, are estimated using training data only and reused for evaluation data.

After preprocessing is selected, the agent considers the available dimension-reduction methods. Selection is based on method assumptions, computational requirements, the type of structure being investigated, and whether held-out transformation is useful. For example, PCA provides a linear global representation and explained-variance diagnostic, whereas UMAP provides a nonlinear view emphasizing local neighborhoods. PCA, Kernel PCA, Isomap, LLE, and UMAP support evaluation transformation in the implemented workflow; MDS, Laplacian Eigenmaps, and t-SNE are treated as training-only. The agent is explicitly instructed not to run every method or select methods solely because known labels appear better separated.

Component	Role
Codex	Planning, decisions, tool use, interpretation, reporting
AGENTS.md	Repository-level workflow and constraints
SKILL.md	Statistical and method-specific guidance
Python scripts	Deterministic inspection, DR, metrics, plots
scikit-learn	Preprocessing, DR methods, trustworthiness
umap-learn	UMAP fitting and transformation
Matplotlib	Visualization

Method settings remain under agent control. Defaults provide reasonable starting points rather than universal choices. When an important result appears sensitive to a setting, the agent may perform a small targeted sensitivity analysis by rerunning an existing tool with a justified alternative setting. Broad hyperparameter searches are avoided. This creates a feedback loop in which observations from one tool call can influence the next action without allowing the computational toolbox itself to change during an evaluation run.

Embedding quality is evaluated using both method-specific and common diagnostics. PCA reports explained and cumulative variance, while trustworthiness provides a common measure of local-neighborhood preservation. Method-specific quantities such as stress, reconstruction error, or KL divergence are retained when applicable but are not treated as directly comparable scores. The agent is also instructed not to declare a method superior solely because it has the highest trustworthiness.

Identifiers and known labels are excluded from unsupervised fitting. Labels may be used only afterward for visualization and interpretation. For methods supporting out-of-sample transformation, models are fitted on training data and then used to transform evaluation observations. These constraints reduce data leakage and make the same workflow applicable to datasets with or without supplied labels.

3 Experimental Evaluation

The final system was evaluated on two structurally different datasets using nearly identical high-level prompts. PathMNIST contains flattened RGB histopathology images, while PBMC3K contains sparse single-cell gene-expression counts. The purpose of the experiments was not to exhaustively benchmark dimension-reduction algorithms, but to examine whether the same agent architecture could make and explain appropriate dataset-dependent analysis decisions.

	PathMNIST	PBMC3K
Training observations	4,500	2,160
Evaluation observations	900	540
Features	2,352 pixels	2,000 genes
Preprocessing	None	Log normalize
Selected methods	PCA, UMAP	PCA, UMAP
PCA 2D variance	57.01%	10.03%
PCA trust., train	0.8059	0.7890
PCA trust., eval	0.8475	0.7850
UMAP trust., train	0.8141	0.7894
UMAP trust., eval	0.8468	0.7804
Targeted follow-up	UMAP $k = 30$	UMAP $k = 30$

Table 1: Autonomous decisions and primary quantitative results. Trustworthiness was evaluated with 10 neighbors. The table summarizes agent behavior rather than ranking the methods.

PathMNIST. The first dataset contained 4,500 training and 900 evaluation images, with each 28×28 RGB image represented by 2,352 pixel features. Inspection showed dense, finite pixel measurements on the same 0–255 intensity scale with relatively similar feature variability. The agent therefore preserved the original representation rather than standardizing individual pixels or applying count normalization. It selected PCA as an interpretable linear baseline and UMAP as a complementary nonlinear neighborhood representation. Kernel PCA, MDS, Isomap, LLE, Laplacian Eigenmaps, and t-SNE were considered but not executed because their additional assumptions, computational costs, overlap with UMAP, or lack of held-out transformation were not justified by the analysis objective.

The first two PCs explained 57.01% of training variance. PCA and UMAP had similar evaluation trustworthiness (0.8475 and 0.8468, respectively), so the agent did not declare a single preferred representation based on this metric. After observing detached regions in the initial UMAP, the agent performed a targeted follow-up by increasing `n_neighbors` from 15 to 30 while holding other settings fixed. The broad structure persisted while smaller geometric details changed, leading the agent to retain the initial setting but caution against interpreting precise island geometry.

PBMC3K. The second dataset contained 2,160 training and 540 evaluation cells with 2,000 selected gene-count features. In contrast to PathMNIST, inspection showed sparse nonnegative integer counts and substantial variation in per-cell totals. The agent selected library-size normalization to 10,000 followed by `log1p`, rather than preserving the raw counts or applying feature standardization alone. This demonstrates that preprocessing was not hard-coded by the pipeline.

The agent again selected PCA and UMAP, but interpreted their roles in the context of expression data. The first two PCs explained only 10.03% of training variance, indicating substantial information beyond two dimensions. PCA and UMAP also produced similar trustworthiness values, providing no basis for a metric-only winner. Separated UMAP regions and unusual evaluation placements prompted the same bounded 15-to-30-neighbor sensitivity check. Broad regional structure remained, but shape and spacing changed. Because no cell-type labels were supplied, the agent appropriately described expression heterogeneity without assigning unsupported biological identities.

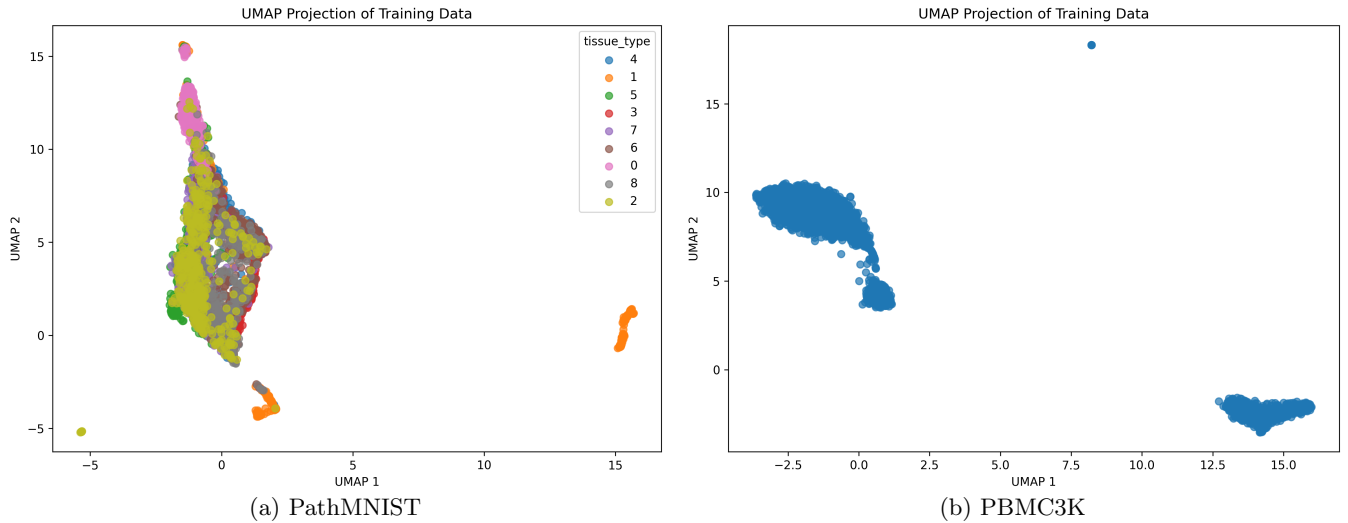


Figure 2: UMAP training embeddings produced by the autonomous analyses. PathMNIST labels are used only for post-hoc visualization; PBMC3K has no supplied cell-type labels.

The contrast between the two experiments provides the main evidence of adaptation. With the same architecture and tool library, the agent preserved PathMNIST pixel intensities but log-normalized PBMC3K counts, while selecting and interpreting methods according to the characteristics observed during each run. In both cases, intermediate results also triggered a targeted follow-up that was not explicitly requested in the initial prompt.

4 Strengths, Limitations, and Conclusion

A primary strength of the system is that preprocessing, method selection, settings, and follow-up analyses are controlled by agent reasoning rather than encoded as a fixed sequence. The modular separation among `AGENTS.md`, statistical skills, and deterministic Python tools makes the reasoning layer extensible while keeping numerical computations reproducible. The two experiments demonstrate adaptation across very different data modalities, and the feedback loop allows the agent to react to intermediate results without resorting to exhaustive parameter searches. Explicit train/evaluation separation and exclusion of identifiers and labels also reduce leakage.

The system has several limitations. First, the agent can choose only among the preprocessing and dimension-reduction operations supplied by the repository; useful analyses outside this toolbox must be reported as limitations rather than implemented during a run. Second, method and hyperparameter selection rely on LLM reasoning rather than a formal optimization criterion, so reasonable alternative decisions may exist. Third, sensitivity analysis is intentionally targeted and does not establish robustness over all hyperparameters or random seeds. Finally, trustworthiness measures only local-neighborhood preservation, while method-specific diagnostics measure different properties. Two-dimensional representations necessarily discard information and may distort global geometry, so visual separation alone cannot establish true clusters or scientific conclusions.

Overall, this project demonstrates a modular AI agent that can translate a high-level dimension-reduction task into an executable and reproducible analysis. Its contribution is not a new dimension-reduction algorithm or an exhaustive comparison of existing methods, but an agent architecture that inspects data, makes and explains dataset-dependent statistical decisions, uses controlled computational tools, evaluates its results, and adapts its analysis in response to intermediate evidence.

References

- [1] Longfei Zhang. BIOS 774 Course Slides. *An Intro to Building Agents in 2026*, 2026.